

Dual Neural Network Framework with SPICE Integration for Fast and Accurate Transistor Modeling

Rodion Novkin* and Hussam Amrouch

Neural network (NN)-based compact transistor models have recently emerged as a promising solution to simplify device modeling. However, they are often deployed and evaluated standalone due to the lack of compatibility with existing simulation program with integrated circuit emphasis (SPICE) software. To investigate the benefits of the NN-based compact models, the proposed framework is integrated into commercial SPICE tool, and NN models' speed is compared with the existing in-built and Verilog-A industry standard implementations. Additionally, the speed-up of NN-based compact models provided by GPU acceleration is demonstrated for variability analysis, and design technology co-optimization with genetic algorithm is explored. For the best trade-off between NN simulation speed and accuracy, the proposed dual-NN structure employs a parameter generator network, representing devices with different transistor geometry, to generate weights for a current/charge prediction network (CPN). In addition to drain voltage V_D and gate voltage V_G , CPN also incorporates environment temperature and achieves 0.797% I_D error with higher than 0.995 R^2 scores for DC characteristics. Moreover, it maintains the speed within SPICE, outperforming Verilog-A Berkeley short-channel insulated gate field-effect transistor model (BSIM), and can simulate up to 18.8 million DC points per second with GPU acceleration.

used industry-standard transistor compact models, such as Berkeley short-channel IGFET (BSIM),^[1,2] serve as feedback for process optimization and rely on physics-based equations to reproduce device behavior. However, developing an accurate analytical solution to model the underlying physics of device operation is often convoluted, time-consuming, and requires substantial domain expertise. Additionally, transistor behavior becomes more complex for modern advanced technological processes, where quantum effects and other challenges attributed to small 3D confined transistor structures start to take place. As a result, analytical-based models may not provide the correct feedback if underlying physics is not well reflected in the model.

One of the most common techniques in DTCO is variability analysis with Monte-Carlo simulations, which model the statistical spread of transistor parameters and analyze their impact on circuit performance. These simulations are often extensive and time consuming, which necessitates the usage of alternative transistor models.

1. Introduction

Accurate and fast transistor models are essential for the semiconductor industry since they are required for design technology co-optimization (DTCO) and circuit simulation. Variations in manufacturing processes, materials, or operating conditions can significantly impact the electrical characteristics of transistors. Thus, joint optimization of design and manufacturing processes is essential for improving circuit performance. Commonly

Neural network (NN)-based compact transistor modeling received significant attention over the last years due to NN's ability to learn complex data patterns. Given enough parameters, NN can approximate any function without the need to learn underlying physical processes, which is incredibly useful for modeling complex transistor behavior. A simplistic nature of matrix multiplication, which is prevalent in NN architectures, provides compact models with the potential to greatly reduce simulation time during transistor analysis or circuit simulation. Additionally, NN-based compact models open up a possibility for running Simulation Program with Integrated Circuit Emphasis (SPICE) simulations on highly parallelizable GPU devices, although it will require some modifications to existing SPICE frameworks.

Several prior works successfully employed NNs for modeling IGFET,^[3,4] GAAFET,^[5] FinFET,^[6] nFinFET, TFET,^[7] a-IGZO TFT,^[8] dual-gate IWO,^[9] FeFET,^[10] and other field-effect transistors,^[11,12] where a significant speed-up over existing conventional models was demonstrated. NN-based models also proved to be capable of meeting the requirements of a practical transistor model by supporting variability analysis, DTCO,^[7,10] switching dynamics,^[10] circuit simulation^[3,4,9] and standard cell library characterization.^[13,14] However, most research on transistor compact models only considered drain voltage V_D and gate

R. Novkin, H. Amrouch
TUM School of Computation, Information and Technology
Chair of AI Processor Design
Munich Institute of Robotics and Machine Intelligence
Technical University of Munich
Munich 85748, Germany
E-mail: rodion.novkin@tum.de

 The ORCID identification number(s) for the author(s) of this article can be found under <https://doi.org/10.1002/aisy.202401085>.

© 2025 The Author(s). Advanced Intelligent Systems published by Wiley-VCH GmbH. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

DOI: 10.1002/aisy.202401085

voltage V_G to produce necessary outputs and did not incorporate temperature effects, which is essential for real-life scenarios since transistors and circuits rarely operate with fixed environmental temperatures. Furthermore, developed NN-based compact models require a proper integration into existing SPICE software without losing their advantages and enabling all benefits provided by SPICE software.

Our Novel Contributions: 1) In this work, we propose a dual NN architecture by extending and simplifying a flexible two-stage NN approach, proposed by Eom et al.^[8] 2) In addition to drain current I_D , we also model drain charge Q_D and source charge Q_S since they are essential for circuit analysis. We explore multiple devices with three process variation parameters: gate length (L), fin height (H_{FIN}), and equivalent oxide thickness (EOT) of 7 nm FinFETs. We demonstrate our dual NN-based compact model's performance in variability analysis, DTCO, transistor, and simple circuit simulations. 3) We integrate our models into commercial SPICE software and perform a comprehensive analysis of simulation speed and accuracy with various NN sizes. 4) Additionally, we explore the speed-up of NN-based compact models for variability analysis provided by GPU acceleration.

2. Our NN-Based Compact Modeling Framework

Section 2.1 describes dataset generation, used devices, and their parameters. Dual-NN structure for compact transistor modeling is outlined in Section 2.2 with the training algorithm given in Section 2.3.

2.1. Dataset Generation

To create a dataset for training and testing our compact modeling framework, we use a 7 nm FinFET model^[15] with variable gate length (L), fin height (H_{FIN}), and equivalent oxide thickness (EOT). Considering the desired transistor geometry range (see Table 1), we generate 125 devices for training/evaluation and 64 devices for testing with various process variation parameter combinations. Our train/evaluation dataset is separated into two parts with 90% data points in the train part and 10% in the evaluation one. In contrast to other works, where a dataset is generated as a whole, and some devices are selected as test devices, none of the three parameter values of our test dataset appear in the training one. This selection allows us to explore NN-based transistor model generalization in a more complicated task since using all three unseen variables in the input will have a more significant impact on the prediction accuracy. Additionally, measurements for transistor modeling are usually available only for a handful of devices with fixed geometry. Thus, one of the most important compact model properties during the early

stages of development is handling variable non-discrete device parameter values. Each device has drain current I_D , drain charge Q_D , and source charge Q_S characteristics with drain voltage V_D and gate voltage V_G varying from 0 to 0.8 V with a step size of 0.01 V. Environment temperature T sweeps the range from 0 to 100 °C with a step of 25 °C. We do not consider a negative voltage region since it is not required for the evaluation of DC characteristics and SPICE simulations used in this paper. Data were obtained using SPICE and its internal BSIM-CMG model. Leakage currents are also included in generated data and considered by the model.

2.2. Dual Neural Network Structure

The dual-NN structure of the proposed model is shown in Figure 1a and consists of two different networks. The structure adopts the principles of a two-stage NN architecture^[8] with significant modifications. Instead of using two sequential networks with a residual connection for I_D , Q_D , or Q_S prediction, we employ a single current/charge prediction network (CPN), which represents a single device and takes V_D , V_G , and T as inputs. Device parameters L , H_{FIN} , and EOT are inputs for a parameter generator network (PGN), which generates weights for the CPN. This approach effectively allows the selection of devices, where CPN represents a device template and PGN is a storage, where settings for the required device can be selected and applied to the template.

Our CPN is a multilayer perceptron (MLP) with only one hidden layer and tanh activation functions to limit the output values of linear layers. One hidden layer is sufficient to cover nonlinearly separable problems (i.e., XOR) and is enough to learn $I_D - V_G$ and charge patterns of a single device. Reduced CPN depth comes with various benefits. First and foremost, it raises the importance of parameters in individual linear layers, which is incredibly useful if NN output is intended to be adjustable. In practice, it means that CPN is able to represent any device after adjusting certain layers instead of the entire network. For the proposed architecture, we determined that only hidden and output layer weights must be modified. Modifying an output layer alone does not provide enough leeway to learn an entire range of devices from Table 1, and modification of the parameters in the input layer does not improve the final result. Additionally, reduced NN depth minimizes the number of nonlinear activations, which is important for fast inference during SPICE simulations. A lower number of layers also prevents vanishing gradients and ensures better gradient flow during the backward pass. Since PGN is effectively attached after the hidden and output layers of CPN, adding more layers in the backward pass will lower the gradient values, especially for the first layers of the PGN as they are deeper in the backward pass.

On the other hand, by minimizing the number of layers, we significantly reduce the number of NN parameters and, as a result, its capability to learn data. To counteract this challenge and ensure that the number of parameters is sufficient for the given task, we make a CPN wider by increasing the number of neurons in the hidden layer. A wider NN benefits more from computation parallelization since matrix multiplication within layers can be done in parallel, and each sequential layer has to

Table 1. Model parameter types and their values for dataset generation.

Parameter	Train/Evaluation	Test
L [nm]	[18, 22, 26, 30, 34]	[20, 24, 28, 32]
H_{FIN} [nm]	[30, 35, 40, 45, 50]	[33, 38, 43, 48]
EOT [nm]	[0.9, 0.95, 1.0, 1.05, 1.1]	[0.92, 0.97, 1.02, 1.07]

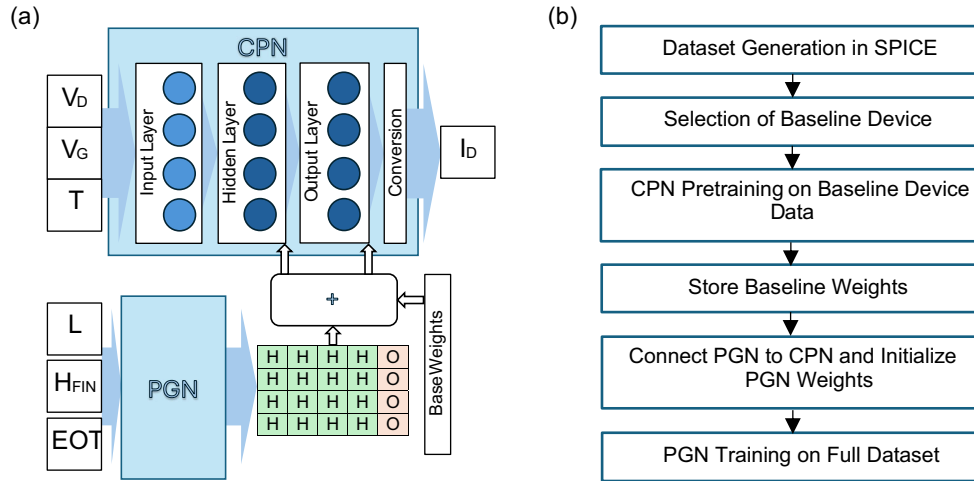


Figure 1. Dual neural network (NN) structure for I_D prediction (left) and its training algorithm (right). The current prediction network (CPN) has one hidden layer with n neurons and represents one device with selected L , H_{FIN} , and EOT . CPN takes V_D , V_G , and T as an input and produces the corresponding I_D . Single parameter generator network (PGN) takes device parameters L , H_{FIN} , and EOT as an input and generates $n \times (n + 1)$ weight differences for hidden (H) and output (O) layers of CPN. These differences are applied to the base weights of a pretrained CPN, thus modifying CPN to reflect the selected device behavior. The dual-NN structure effectively stores multiple device models in PGN, while each device model remains small and maintains remarkable speed and accuracy. NN structures for Q_D and Q_S are identical. a) Dual-NN structure; b) Dual-NN training algorithm.

wait for the output of the previous layer to continue the calculation. The number of neurons in our experiments varies since we perform a comprehensive analysis on CPN execution speed. Table 2 compares our CPNs with different sizes with the NNs used in ref. [8] for current prediction. However, our CPNs are designed to incorporate temperature in addition to voltages, and we employ only one PGN per modeled characteristic instead of multiple to generate weights. The output of the CPN is a single value, and three different networks are used for I_D , Q_D , and Q_S . Similar to previous works,^[3,4,8] we also employ conversion functions to limit and modify the output range of the CPN. Our conversion functions for I_D , Q_D , and Q_S for the corresponding NN outputs y_I and y_Q are:

$$I_D = V_D \cdot e^{y_I}, \quad Q_{D,S} = y_Q \cdot 10^{-18} \quad (1)$$

Conversion function for I_D can determine the output sign of the drain current without the need to learn it with a CPN. As mentioned previously, a negative I_D region is not explored in this work since it is not needed for the experiments. However, preserving the deterministic sign functionality in I_D models is essential for potential future framework extensions. Additionally, the I_D conversion function allows training in a logarithmic scale, which is necessary for the current. Since the charge sign cannot be determined easily through V_D , all charges are just scaled to a region where networks can operate efficiently.

Table 2. Number of multiplication–additions for one DC point in linear layers of proposed CPNs and NNs from ref. [8].

Neurons	8	16	32	BMN + CPN from ref. [8]
Mult-Adds	113	353	1217	171

In contrast to CPN, inference time for PGN is less relevant since device parameter generation happens once per device. Nevertheless, we avoid using complex NN architectures and employ an MLP with two hidden layers and tanh activation functions. Instead of using device parameters as a single vector for PGN input, we embed each device parameter with a linear layer, which has n_e neurons. Thus, hidden layers have $3n_e$ neurons since embeddings of all three device parameters are concatenated. Embedding effectively maps one value into a higher-dimensional vector allowing PGN to learn the effect of each device parameter separately in the earlier layers. The output of the PGN is a matrix with a size of $n \times (n + 1)$, where $n \times n$ and $n \times 1$ are weight differences for a hidden and an output layer of CPN, respectively (see Figure 1a). These weight differences are applied to the baseline CPN weights, adjusting its output. Note that biases of CPN linear layers are not modified to avoid unnecessary complexity with PGN training, since output adjustment of individual layers can already be achieved with the weight modification alone.

The main advantages of a dual-NN approach and PGN are in inference and the intended area of its application. Since the goal is to model transistor behavior, our approach heavily exploits the similarity of the output current and charges for various transistor geometry parameters. Additionally, during circuit simulation, it is necessary to maintain high simulation speed, and transistor geometry is usually fixed, while T , V_D , and V_G require full operational range. Thus, naturally it is highly beneficial to separate these characteristics to achieve high simulation speeds with as small as possible CPN. Having all input parameters considered within one NN will result in a much higher number of required layers and neurons. Thus, processing these input parameters constantly is not efficient if they are not changing. PGN memorizes parameter configuration for all devices and enables device selection, effectively separating constant and single-time calculations during inference.

2.3. Neural Network Training Algorithm

The training algorithm for the proposed dual-NN framework is shown in Figure 1b. In the initial step, a complete dataset of I_D , Q_D , and Q_S characteristics is generated for all devices in Table 1. The first phase of the training process starts with the selection of the baseline device, which can be arbitrarily chosen from the available devices in the training dataset. We selected the device in the middle of the spectrum to balance the difference in I_D , Q_D , and Q_S characteristics. Our selected baseline device has the following process variation parameters: $L = 34$ nm, $H_{FIN} = 40$ nm, and $EOT = 1$ nm. In the next step, a CPN is trained using the data of the selected baseline device. The training is done in PyTorch^[16] with an Adam optimizer. An initial learning rate is reduced during training every time the loss value reaches a plateau. To increase training stability, we minimize the number of weight updates in one epoch and use batch size equal to the number of all data points in the training dataset. Similar to previous approaches,^[3,4,8] we incorporate gradient information to achieve the best fitting for the transistor model. Our loss function for I_D CPN training in the first phase employs mean squared error (MSE) and is defined as:

$$L_I = a \cdot \text{MSE}(I_D) + \text{MSE}(\log(I_D)) + \text{MSE}(g_m) + \text{MSE}(g_{DS}) + \text{MSE}(g'_m) + \text{MSE}(g'_{DS}) \quad (2)$$

$$\text{MSE}(Y) = \frac{1}{p} \sum_{i=1}^p (y_i - \hat{y}_i)^2 \quad (3)$$

where parameter a is used to boost the influence of the current in the saturation region and equals 100 in all experiments. We do not utilize derivatives over temperature since the step resolution of 25 °C is too high for an accurate derivative estimation. Our loss function for Q_D and Q_S is:

$$L_Q = \text{MSE}(Q) + \text{MSE}(\partial Q / \partial V_G) + \text{MSE}(\partial Q / \partial V_D) \quad (4)$$

In contrast to other works, we avoid extensive parameter search and do not perform iterative training. Parameter a is selected to equalize the influence of various parts in Equation (2) and remains the same for NN architectures with different numbers of neurons n . CPN is trained in one iteration for 30 000 epochs. We perform gradient clipping to avoid gradient explosion due to extremely high loss values at the beginning of the training since it can prevent NN from learning properly. It is crucial to achieve the best possible performance for the baseline device since the performance of the entire dual-NN framework heavily depends on the quality of base weights obtained in the CPN training phase. However, as mentioned previously, we still perform some adjustments to our loss function to simplify the training procedure without sacrificing too much accuracy. Pretrained weights of the CPN for each characteristic are stored and used as base weights during PGN training in the next step.

Weight initialization is an important step, which influences the success of PGN training. CPN output is extremely sensitive toward any weight changes in linear layers. To avoid huge error spikes at the beginning of the PGN training, we initialize PGN

weights from a normal distribution with zero mean and variance $\sigma = 0.2$. This weight initialization guarantees a smooth start of the training since PGN influence on the pretrained CPN will be minimal. During the training, weight differences, generated by PGN, are applied to linear layers of CPN and I_D , Q_D or Q_S output error is propagated back to PGN. Since only minor adjustments are required in the PGN training phase, we simplify our loss functions even further by removing derivative terms and parameter a , which results in the following I_D loss for the second training phase:

$$L_I = \text{MSE}(I_D) + \text{MSE}(\log(I_D)) \quad (5)$$

Charge loss remains the same as in Equation (4) to ensure the correct behavior of the derivatives for trained PGNs. Due to very high CPN sensitivity, we also use a small initial learning rate $lr = 0.0002$. PGN is trained for 1000 epochs using Adam optimizer, and the learning rate is halved every 80 epochs. To ensure smooth training and minimize the number of parameter updates every epoch, we employ gradient accumulation every 10 batches since the dataset size does not allow the one-batch approach, which was previously used for CPN training. Since I_D loss is simplified, PGN training requires around 6 h on an RTX3070 GPU. On the other hand, charge training needs 11 h per PGN since derivative calculations add some computational load.

To ensure smooth training and achieve the best performance, we also implement certain input data transformations. Voltages V_D and V_G were left as is and not normalized since they are directly used in the conversion function (see Equation (1)), and for derivative calculation in loss functions (see Equation (2) and (4)). Additionally, further framework extension will likely incorporate negative voltage regions, which will naturally center all voltages around zero. EOT range of [0.9, 1.1] is already rather small and does not require scaling or centering. All other input variables were normalized based on their initial range to increase training stability:

$$T^{\text{norm}} = T/100 - 0.5, \quad H_{FIN}^{\text{norm}} = (H_{FIN} - 40)/10, \quad L^{\text{norm}} = (L - 26)/8 \quad (6)$$

3. Results

Experimental results for the current prediction NN-based model and its suitability for variability analysis are given in Section 3.1. Section 3.2 demonstrates the applicability of the model for DTCO. Section 3.3 shows the accuracy of NN-based models for charge prediction. Multicollinearity analysis for the selected dual-NN architectures is performed in Section 3.4. Benefits of GPU acceleration of NN-based models and speed analysis of integrated to SPICE models are presented in Section 3.5. Results for the transient analysis of a single transistor and inverter are shown in Section 3.6.

3.1. $I_D - V_G$ Accuracy for Variability Analysis

We train multiple current prediction networks and corresponding PGN models with various numbers of neurons n to explore

the correlation between NN-based model performance and its size. Since bigger NNs are more sensitive toward learning rate, we use a slightly different initial learning rate for all CPNs, ranging from 0.005 for the smallest model with $n = 8$ down to 0.003 for the biggest model with $n = 32$. To evaluate a prediction accuracy on such a wide range of data as I_D , we employ a ratio-based MAPE for a predicted value $\hat{y}$ and a true value y :

$$\text{MAPE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|} \quad (7)$$

Results from **Table 3** demonstrate that in the case of CPN, a larger model tends to stagnate in terms of performance. While 8-neuron NN, showing 1.54% MAPE, is not enough for the given complexity of the data, 0.625% error for 32 neurons does not exhibit a significant benefit over a 16-neuron CPN with 0.663% error. PGN training for the dual-NN model reduces the error even further for the evaluation dataset with devices from Table 1 since CPN weights are adjusted to minimize the error. For the 8-, 16-, and 32-neuron NNs the MAPE was reduced by 0.037%, 0.052%, and 0.075% correspondingly. Thus, if the intended use of the NN-based model is limited to a group of selected devices, the dual-NN approach can reduce the error of individual CPNs for each device. For a more generalized usage in the case of the test dataset, where all three device parameters are different, MAPE is slightly higher than for the baseline CPN in all cases except a model with $n = 8$, since higher errors tend to fluctuate more and NNs with fewer parameters generalize better. In all other cases, this behavior is expected due to an increased complexity of the data caused by three unseen input variables. However, even though the test MAPE for the smallest dual-NN was lowered to 1.482%, it is still higher than 0.840% for a 16-neuron dual-NN or 0.797% for a 32-neuron one.

For the evaluation of $I - V$ and DC characteristics, we selected a 16-neuron dual-NN model since it provides the best size-accuracy trade-off. Curve examples for various devices and temperatures are shown in **Figure 2**. Lines depict the dual-NN model output, while same-colored squares show the output of SPICE industry-standard model with a 0.1 V step. V_G/V_D in the figure legend means either V_G or V_D is fixed in applicable cases. Figure 2a,b demonstrate an excellent fitting for $I_D - V_G$ and $I_D - V_D$ curves. First-order current derivatives in Figure 2c,d are smooth and consistent. They resemble the correct behavior and match SPICE data well.

Evaluation of DC characteristics is performed for the same 16-neuron dual-NN model. We compare threshold voltage V_{TH} , subthreshold swing SS , on current I_{ON} , and off current I_{OFF} logarithmic value. **Figure 3** demonstrates an excellent match for all four DC characteristics with the R^2 value exceeding 0.995

Table 3. Current mean absolute percentage error (MAPE) for various CPN and dual-NN models.

Number of neurons	8	16	32
CPN MAPE [%]	1.540	0.663	0.625
Dual-NN evaluation MAPE [%]	1.503	0.611	0.550
Dual-NN test MAPE [%]	1.482	0.840	0.797

in all cases, meaning that a dual-NN compact model is capable of learning transistor current behavior with great precision, allowing accurate variability analysis.

3.2. NN-Based $I_D - V_G$ Model for Design Technology Co-optimization

Since our NN-based $I_D - V_G$ model can perform accurate and fast DC characteristics predictions (see Figure 3), we employ it for DTCO. We formulate the task as follows: for a desired value y of threshold voltage V_{TH} , subthreshold swing SS , on current I_{ON} or off current I_{OFF} find device parameters L , H_{FIN} , EOT and environment temperature T . Optimization is performed using a genetic algorithm.

The algorithm is briefly described in **Algorithm 1** using a desired threshold voltage V_{TH} value y as an example. Constraints can include necessary limitations for device parameters. Number of iterations N_{iter} and number of devices (parents) per iteration N_{dev} can be selected arbitrarily. 1) The initial device population is initialized randomly as a 64-bit binary vector X for each parent where the parameter values x_p of EOT, H_{FIN} , L , T are decoded from a 16-bit vector X_p as follows:

$$x_p = x_p^{\min} + \frac{\text{int}(X_p)(x_p^{\max} - x_p^{\min})}{2^{16}} \quad (8)$$

$$X = \text{concatenate}(X_{EOT}, X_{H_{FIN}}, X_L, X_T) \quad (9)$$

where $\text{int}(X_p)$ is an integer representation of a 16-bit vector X_p , $x_p^{\min}$, and $x_p^{\max}$ are constraints (boundaries) of the value x_p . The initial best device is selected randomly, and its corresponding score is evaluated. 2) The optimization process starts with the evaluation of the initialized random population. Evaluation is done with our dual-NN compact model, which produces $I_D - V_G$ curves used for DC characteristics calculation. Scores are calculated as an absolute difference between the desired value y and parent value y_i for the DC characteristic intended for optimization (V_{TH} in the example). 3) Next, the best candidates are selected from the population, and the best device is reassigned. Candidate selection is performed by score comparison between each device $i \in [1, N_{dev}]$ with $k = 3$ other random devices from the population. Note that the best device is not used directly during selection and is mainly employed to track the algorithm's progress. 4) After selection, a one-point crossover is performed between two devices, meaning that the first n bits from one device are swapped with the corresponding bits from another device with a probability $P_{cross} = 0.9$. 5) Finally, the mutation is performed on all devices by performing a bit flip in each element of the device bit vector with a probability $P_{mut} = 0.02$. The output of the algorithm is the parameters of the best device. Since the algorithm is primarily written for a CPU, we also deploy our dual-NN compact model on a CPU to avoid data movement between devices in PyTorch. The total runtime for $N_{iter} = 50$ and $N_{dev} = 100$ on Intel Core i9-13900 is under 10 s, but the best solution is usually finalized within the first 35 iterations. The solution can be verified in SPICE, and in our experiments, the relative error for the desired value does not exceed 1%, which is an expected outcome considering the precision shown in Figure 3.

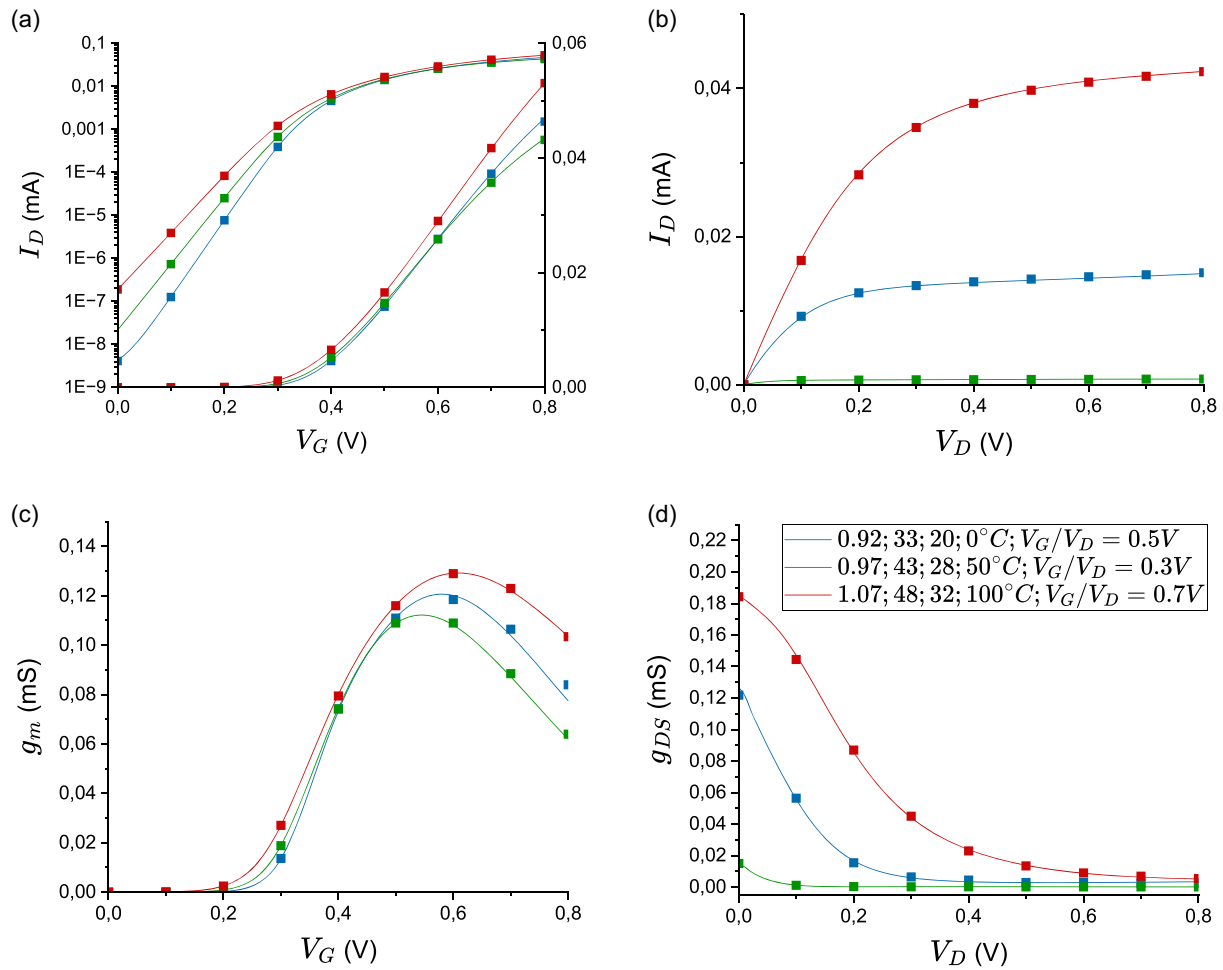


Figure 2. A 16-neuron dual-NN model exhibits an excellent fitting for transistor current and transconductance for various devices with different geometry and environment temperatures. Lines are NN outputs, same-colored squares—SPICE industry-standard compact model. Values in the legend (in order): EOT (nm); H_{FIN} (nm); L (nm); T ($^{\circ}\text{C}$); fixed V_G , or V_D (V), depending on the plot. a) Drain current $I_D - V_G$; b) Drain current $I_D - V_D$; c) Transconductance g_m ; d) Output conductance g_{DS} .

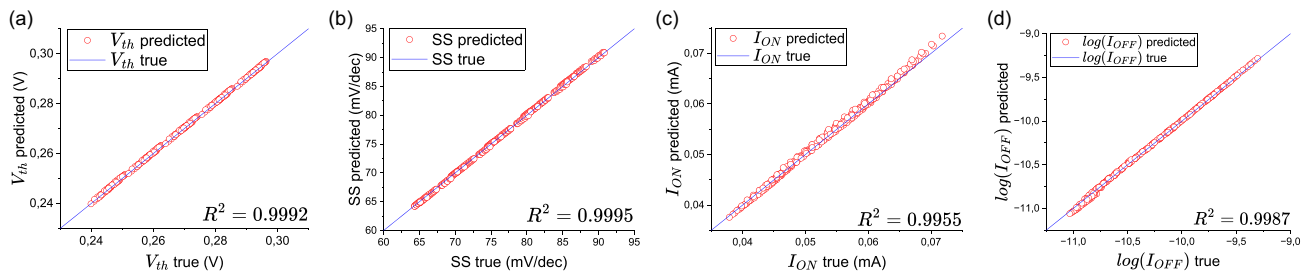


Figure 3. R^2 plots for a 16-neuron dual-NN model demonstrate an excellent accuracy for various transistor DC characteristics compared to the golden industry-standard compact model in SPICE. a) Threshold voltage; b) Subthreshold swing; c) ON current; d) OFF log-current.

Our dual NN-based compact model demonstrated its feasibility for DTCO tasks where it can be used as a fast evaluation platform. Additionally, the model is highly flexible and can be easily employed by more complex multiobjective optimization algorithms, including other openly available ones.

3.3. Q – V Accuracy

Similar to the previous experiment in Section 3.1, we train multiple charge prediction networks and corresponding PGN models while using the same numbers of neurons $n \in [8, 16, 32]$ for

Algorithm 1. Optimization algorithm.

Require: y , constraints, N_{dev} , N_{iter} , P_{cross} , P_{mut}

Ensure: $V_{th} = y$, constraints

for all $i \in [1, N_{dev}]$ **do**

$X_{EOT}^i, X_{H_{FIN}}^i, X_L^i, X_T^i \leftarrow \text{random}()$ ▷ Initialization

$BEST \leftarrow POP[0]$ ▷ Initial selection of best device

$BEST\ SCORE \leftarrow \text{evaluate}(\text{decode}(POP[0]))$

while $iter \leq N_{iter}$ **do**

$SCORE \leftarrow \text{evaluate}(\text{decode}(POP))$ ▷ Device scores

for all $i \in [1, N_{dev}]$ **do**

if $SCORE[i] \leq BEST\ SCORE$ **then**

$BEST \leftarrow POP[i]$

$BEST\ SCORE \leftarrow SCORE[i]$

$POP \leftarrow \text{select}(POP)$ ▷ Select best candidates

$POP \leftarrow \text{crossover}(POP, P_{cross})$

$POP \leftarrow \text{mutate}(POP, P_{mut})$

$EOT, H_{FIN}, L, T \leftarrow \text{decode}(BEST)$

return EOT, H_{FIN}, L, T

simplicity. In contrast to the current, the charge network is trained in a linear scale (see conversion functions in Equation (1)). Therefore, very small output values, e.g., lower than 0.01, can significantly influence a ratio-based metric since NNs do not generally work well with outputs of this scale. For instance, $\hat{y} = 0.002$ and $y = 0.001$ in Equation (7) yield a 100% error, which could disproportionately skew the average MAPE. Thus, we use kernel density smoothing to demonstrate the accuracy of the selected 16-neuron dual-NN compact model. **Figure 4** shows distributions for all data points from the test dataset. In both cases, data matches perfectly, with the bandwidth values w being extremely close. However, Q_S NN prediction in **Figure 4b** has a slightly higher bandwidth deviation from the SPICE industry-standard compact model compared to the Q_D case in **Figure 4a** (0.00212 and a 0.0009 difference in w respectively). Slightly worse performance for Q_S can be explained by a harder data pattern with a noticeable skew near zero. To evaluate the quality of the trained dual-NN charge models further, we look at the derivatives of Q_D and Q_S . **Figure 5** demonstrates that derivatives in both cases are smooth, consistent, and match out-put well.

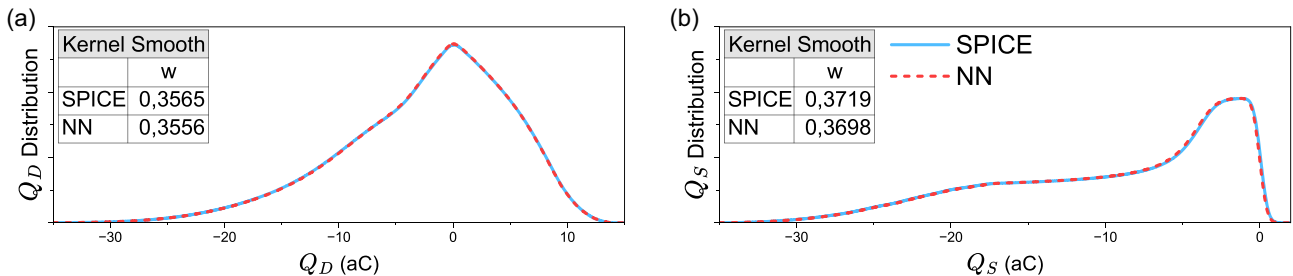


Figure 4. Charge distributions for a 16-neuron dual-NN model (blue) demonstrate an excellent match with SPICE golden industry-standard compact model (red) on the test dataset. a) Drain charge, Q_D ; b) Source charge, Q_S .

3.4. Multicollinearity Analysis

Since we employ two NNs in our framework with both of them contributing to the total number of parameters in the dual-NN model, it is important to avoid overparameterization to achieve the best generalization and fitting for the given dataset. Thus, before getting to the speed test, we would like to perform multicollinearity analysis for both, data and NN weights of the selected model configurations, and ensure that NN architectures were chosen correctly.

Multicollinearity analysis for data aims to uncover input variables or features which are highly correlated. These dependent on each other variables can lead to a significant performance degradation for NNs and reduce their learning capability. However, transistor modeling task completely excludes the possibility of highly correlated data appearing in the model since all input variables are defined physical properties of a transistor. They are all independent from each other by definition and each variable has its corresponding effect on the current or charge, which excludes data correlation in the output either. Thus, multicollinearity analysis for data is not necessary in the case of transistor modeling.

Multicollinearity in NN architectures is often referred as weight redundancy or over-parameterization. One of the most common tests to determine whether a model has redundant weights or not is to analyze correlation between weights in NN layers. Table 3 demonstrates that 8-neuron NN is significantly underperforming and is clearly under-parameterized. For further investigation, we select a 16-neuron architecture and analyze weights in hidden layers of I_d CPN and its corresponding PGN. **Figure 6a** shows a noticeable correlation in some weights of the CPN, meaning that the 16-neuron CPN is slightly over-parameterized for the fitting of the single device $I_D - V_G$ curve. However, this weight redundancy in CPN is beneficial for the general task since CPN weights are intended to be modified to fit a much wider range of data and having redundant weights gives PGN more flexibility to manipulate them. In contrast to CPN, PGN weights in **Figure 6b,c** are not correlated and show no sign of overparameterization, meaning that the dual-NN architecture does not have weight redundancy and PGN architecture can potentially be improved by increasing the number of its parameters. Notably, 16-neuron charge CPN and its corresponding PGN follow the same pattern. Multicollinearity analysis for a 32-neuron dual-NN architecture is not shown since its small accuracy gain is not practical due to the significant inference

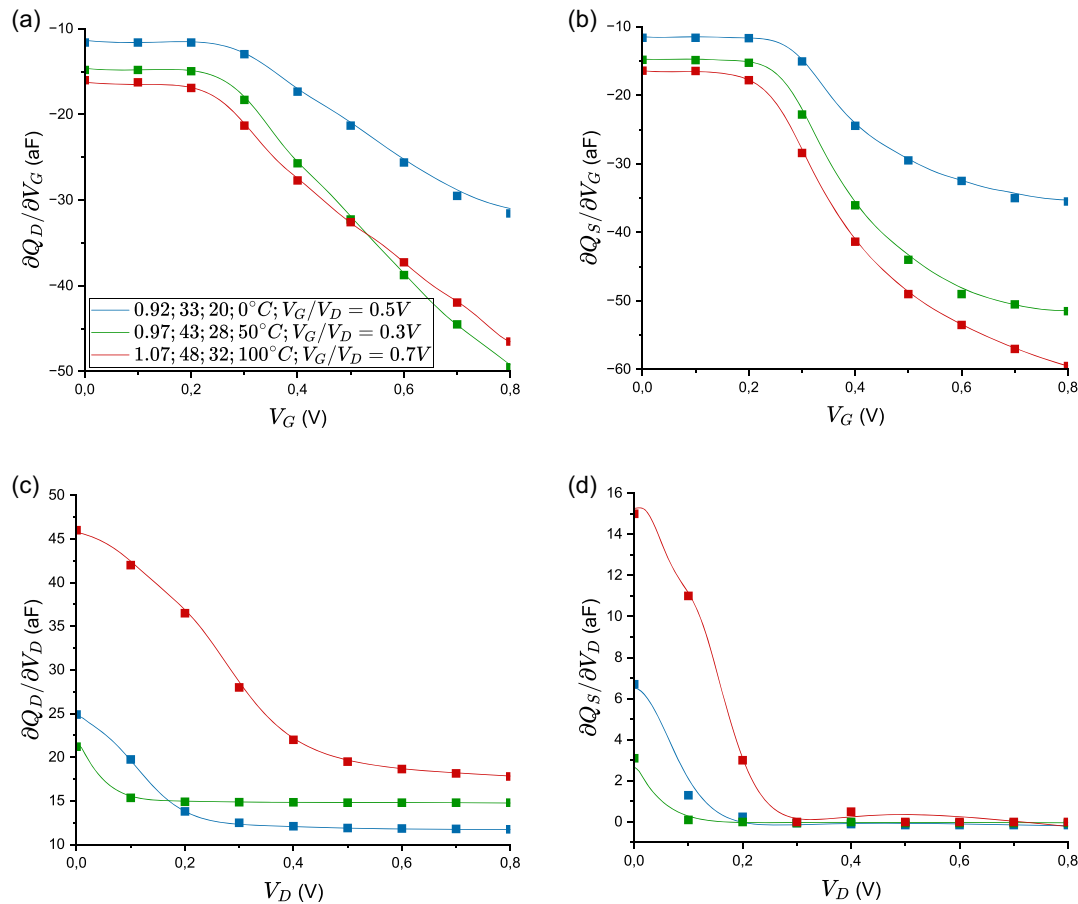


Figure 5. Charge derivatives for a 16-neuron dual-NN model (lines) show a good match with SPICE industry-standard compact model (same-colored squares). Values in the legend (in order): EOT (nm); H_{FIN} (nm); L (nm); T (°C); fixed V_G ; or V_D (V), depending on the plot. a) $\partial Q_D/\partial V_G$; b) $\partial Q_S/\partial V_G$; c) $\partial Q_D/\partial V_D$; d) $\partial Q_S/\partial V_D$.

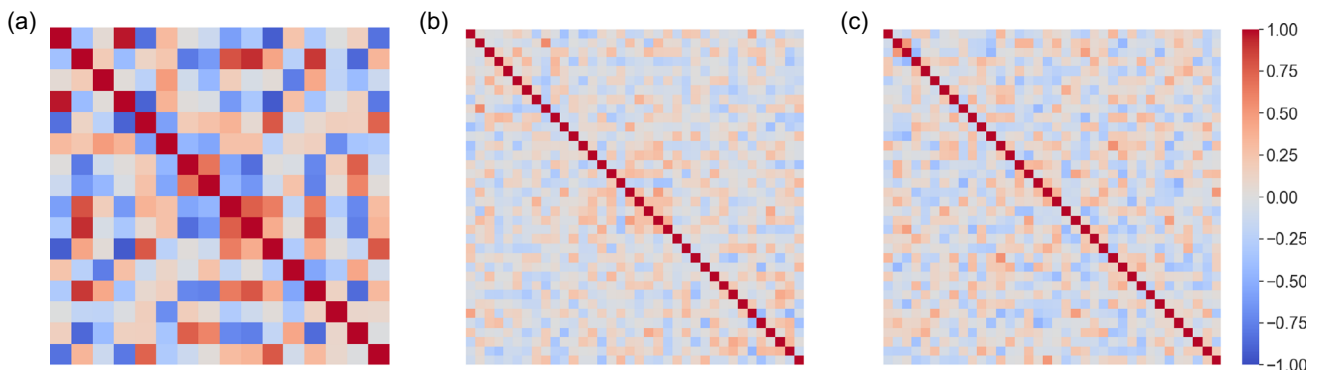


Figure 6. Correlation analysis of weights in hidden layers of CPN and PGN. Correlation values close to 1 or -1 represent high correlation and indicate some weight redundancy. Over-parameterization in CPN is beneficial since its weights are intended to be modified giving PGN more flexibility to manipulate them and ensure better fitting for different transistor geometry. a) CPN hidden layer; b) PGN first hidden layer; c) PGN second hidden layer.

speed reduction, which will be demonstrated later in Section 3.5. However, despite having slightly more correlated weights in 32-neuron CPNs, their respective PGNs still do not exhibit any signs of weight redundancy.

3.5. Impact of Model Size on Simulation Speed

To demonstrate the potential speed advantage of the proposed compact modeling framework for variability analysis outside

of SPICE, we code the $I - V$ transistor model in C and CUDA. The C code speed is tested on Intel Core i9-13900 CPU and does not employ multi-threading or parallelization. The speed of the CUDA code is measured on 8Gb NVIDIA GeForce RTX 3070. To detach execution speed from the specific NN architecture, we provided the number of multiplications and additions required for the calculation of one DC point by linear layers of various CPNs in Table 2.

Figure 7a demonstrates that the C-code model of the proposed CPN architecture is up to 56 times faster than an internal SPICE industry-standard compact model. CPN models with a higher number of neurons in a hidden layer have lower speeds due to the increased number of arithmetic operations during NN inference. Nevertheless, bigger models with 16 and 32 neurons can still achieve a $23\times$ and $7\times$ speed-up over SPICE correspondingly. For reference, we also added to Figure 7a the $I - V$ NN execution speed from Tung et al.^[4] However, our model speed is measured on a different CPU and includes temperature. In contrast to the C implementation, the CUDA code of the proposed CPNs utilizes all the benefits of GPU multi-threading. The smallest CPN with 8 neurons in a hidden layer can simulate 18.8 million DC points per second and achieves 357 times faster DC simulation with GPU acceleration compared to the internal SPICE industry-standard compact model. The biggest CPN with 32 neurons in a hidden layer still outperforms any other non-GPU implementation, reaching 5.6 million DC points per second, which demonstrates the feasibility of bigger NN architectures, which can potentially be employed for more complex transistors.

Even though external NN implementations show outstanding performance, the preferable way of using a compact model is still with an existing SPICE framework. We integrate our $I - V$ and $Q - V$ CPN implementations into the previously used commercial SPICE software to compare the speed within one SPICE framework, where computational overhead will be the same for all approaches. Verilog-A is a common method for importing externally developed models, but it cannot perform matrix multiplications efficiently, which results in a significant performance loss of any integrated NN. Since GPU-specific code cannot be used within SPICE either, we integrated our $I - V$ and $Q - V$ CPN C-code implementations. Figure 7b demonstrates the

performance of the proposed CPN architectures within the SPICE framework. Note that the number of neurons in Figure 7b is the same for all CPNs, meaning that the number of multiplication additions in Table 2 during SPICE execution is tripled since there are three networks. To ensure a fair comparison, all simulation parameters except a transistor model remain the same. The SPICE simulation uses four threads in all cases, and the speed was measured on Intel Core i9-13900 CPU. Compared to the previous experiment (see Figure 7a), where SPICE computational overhead was not considered for NNs, CPN execution speed was greatly reduced. However, an integrated NN-based compact model with 8 neurons in a hidden layer achieves 56.1 thousand DC points per second (kDCp s^{-1}) and still slightly outperforms an internal SPICE industry-standard compact model with 52.6 kDCp s^{-1} . Even though an 8-neuron model did not demonstrate sufficient accuracy in our experiments, it can still be employed in the cases without the temperature input. Compared to the commonly used Verilog-A BSIM-CMG compact model with 24.5 kDCp s^{-1} , NN-based implementation achieves higher speed even in the case of the largest model with 32 neurons, which reaches 24.7 kDCp s^{-1} . Therefore, NN-based compact transistor models are a viable option for existing commercial SPICE software.

3.6. Accuracy of Our NN-Based Transistor Model for Transient Analysis

We use integrated SPICE transistor models to test their applicability for SPICE circuit simulation. Since we only have Q_D and Q_S , NN models and all four charges are required for analysis, and we fix the value of bulk charge $Q_B = 1.46 \times 10^{-20}$. For the 7 nm fin field-effect transistor (FinFET), this value is rather static and small compared to other charge values. Therefore, the gate charge Q_G can be calculated as:

$$Q_G = -(Q_D + Q_S + Q_B) \quad (10)$$

In the first experiment, we apply a V_G pulse to a single transistor and measure the drain current at each time step. The analysis is performed at $T = 40^\circ\text{C}$. **Figure 8a** shows matching current curves for SPICE and NN-based compact model outputs with a

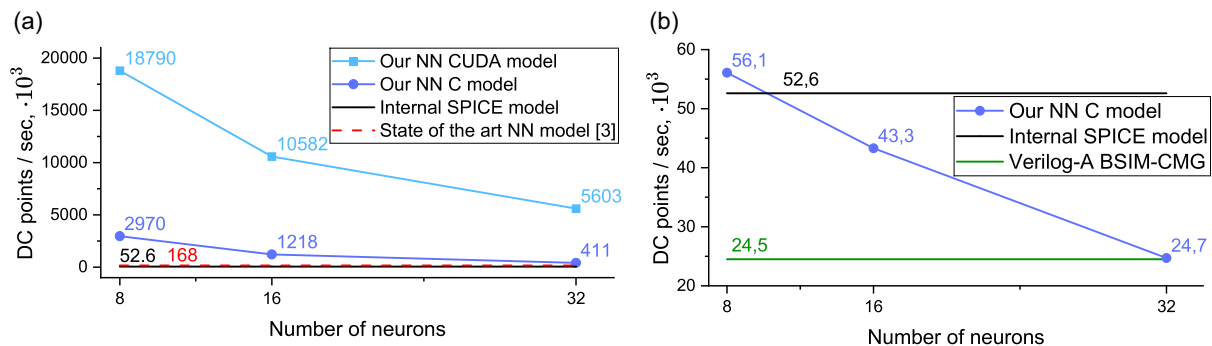


Figure 7. DC simulation time comparison between internal and Verilog-A SPICE industry-standard compact models and our NN-based compact models with different sizes. All NN models show remarkable simulation speed outside of SPICE, especially with GPU acceleration. After integration, the major speed benefit of NNs is not retained due to the SPICE calculation overhead. However, all NNs still show higher simulation speeds than Verilog-A BSIM-CMG industry-standard compact models in SPICE. a) NNs are outside of SPICE framework; b) NNs are integrated into SPICE software.

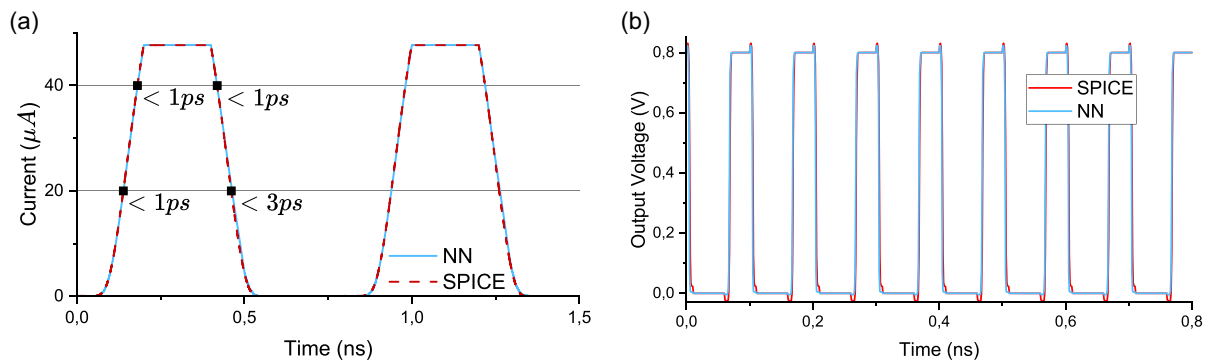


Figure 8. Transient analysis of a single transistor (left) and inverter (right) using 16-neuron dual-NN models shows a good match with SPICE simulation with industry-standard compact model. a) Single transistor; b) Inverter.

time difference of less than 1ps in the majority of points. To demonstrate the applicability of the model for simple circuit simulation, we perform a transient analysis of an inverter, applying a V_G pulse and setting $T = 60^\circ\text{C}$. Figure 8a shows that an NN model has a correct slope but exhibits a more simplified behavior during switching. In both cases, simulations converged without any issues.

4. Conclusion

We presented a dual-NN transistor compact modeling framework, which achieved a 0.797% I_D error and excellent fitting for Q_D and Q_S compared to the golden industry-standard compact model. Additionally, the proposed dual-NN model was able to achieve high accuracy in variability analysis, exhibiting higher than 0.995 R^2 scores for DC characteristics and allowing fast and reliable DTCO. Furthermore, our model captured well the behavior of current and charge derivatives, which is necessary for SPICE simulations. While being sufficient for transistor and simple circuit simulation, our NN-based models, integrated into commercial SPICE software, retained the benefits of NN-based compact modeling, showing higher simulation speeds compared to industry-standard and Verilog-A BSIM-CMG models. Finally, we demonstrated the benefits and vast potential of GPU acceleration for fast and accurate transistor variability analysis.

In future work, we will continue expanding our model and building an end-to-end framework to enable the automated creation of NN-based compact models from the measurement data and their further integration to SPICE software.

Acknowledgements

This work was supported in part by the Federal Ministry of Education and Research (16ME0995). The authors would like to thank Anirban Kar for the valuable support and discussions regarding the modeling of transistors. Open Access funding enabled and organized by Projekt DEAL.

Conflict of Interest

The authors declare no conflict of interest.

Author Contributions

Rodion Novkin: investigation (lead); methodology (lead); software (lead); writing—original draft (lead). **Hussam Amrouch:** supervision (lead).

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Keywords

circuit simulation, design technology co-optimization, machine learning, neural network, transistor compact model

Received: December 11, 2024

Revised: March 18, 2025

Published online: April 21, 2025

- [1] Y. S. Chauhan, S. Venugopalan, M.-A. Chalkiadaki, M. A. U. Karim, H. Agarwal, S. Khandelwal, N. Paydavosi, J. P. Duarte, C. C. Enz, A. M. Niknejad, C. Hu, *IEEE Trans. Electron Devices* **2014**, 61, 234.
- [2] J. P. Duarte, S. Khandelwal, A. Medury, C. Hu, P. Kushwaha, H. Agarwal, A. Dasgupta, Y. S. Chauhan, *ESSCIRC Conf. 2015-41st European Solid-State Circuits Conference (ESSCIRC)*, Graz, Austria, September **2015**, pp. 196–201.
- [3] C.-T. Tung, M.-Y. Kao, C. Hu, *IEEE Trans. Electron Devices* **2022**, 69, 6476.
- [4] C.-T. Tung, C. Hu, *IEEE Trans. Electron Devices* **2023**, 70, 2157.
- [5] M.-Y. Kao, H. Kam, C. Hu, *IEEE Electron Device Lett.* **2022**, 43, 974.
- [6] K. Sheelvardhan, S. Guglani, M. Ehteshamuddin, S. Roy, A. Dasgupta, *IEEE Trans. Electron Devices* **2024**, 71, 239.
- [7] Z. Zhang, R. Wang, C. Chen, Q. Huang, Y. Wang, C. Hu, D. Wu, J. Wang, R. Huang, *2019 Silicon Nanoelectronics Workshop (SNW)*, Kyoto, Japan, June **2019**, pp. 1–2.
- [8] S. Eom, H. Yun, H. Jang, K. Cho, S. Lee, J. Jeong, R. Baek, *Adv. Intell. Syst.* **2024**, 6, 2300435.
- [9] G. Choe, J. Kwak, S. Yu, *IEEE Trans. Electron Devices* **2024**, 71, 231.
- [10] S. Thomann, R. Novkin, J. Li, Y. Hu, J. Xiong, H. Amrouch, *IEEE Trans. Electron Devices* **2024**, 71, 213.
- [11] J. Wang, Y.-H. Kim, J. Ryu, C. Jeong, W. Choi, D. Kim, *IEEE Trans. Electron Devices* **2021**, 68, 1318.
- [12] W. Jang, S. Myung, J. M. Choe, Y.-G. Kim, D. S. Kim, *IEEE Electron Device Lett.* **2023**, 44, 1368.

- [13] F. Klemme, Y. Chauhan, J. Henkel, H. Amrouch, in *Proc. 39th Int. Conf. on Computer-Aided Design*, Association for Computing Machinery, Virtual Event, USA, October **2020**.
- [14] F. Klemme, H. Amrouch, *IEEE Trans. Circuits Syst. I* **2021**, 68, 2569.
- [15] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, G. Yeric, *Microelectron. J.* **2016**, 53, 105.
- [16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, *Advances in Neural Information Processing Systems* 32, Curran Associates, Inc, San Jose, CA, USA **2019**, pp. 8024–8035.